

# Sentinel Command

Technical Proposal — High-Level Design for the Gujarat Police Innovation Challenge 2026 (Sentinel CCTV). Author: Nikhil. Scope: Hybrid **Model 1** (central CCTV registry and GIS) + **Model 2** (unified viewing, ANPR metadata, watchlist alerts, cross-camera hunt).

*Sentinel Command unifies the official Sentinel camera catalogue on a GIS map, plays live feeds in one dashboard, runs ANPR with watchlist alerts, and produces hunt trails with CSV export — built to the official cameras.json, RTSP-TCP, and HLS contract on cctv.corp8.cloud.*

## 1. Problem and permitted approach

Twenty-six departments operate independent CCTV estates across Gujarat — analog and IP, cloud and local storage, different vendors and retention. Police lack a single map of assets, a single viewer, and a way to correlate live video with stolen/wanted records (VAHAN, SARTHI, eGujCop). The official brief allows any reference model, a hybrid, or a custom architecture. Model 1 is the mandatory GIS foundation.

**Choice:** Hybrid Model 1 + 2. Justification for Category 1 (solo): deliver a working registry and a working ANPR/hunt loop on the sandbox feeds, without faking Model 3 vendor adapters or a Model 4 central VMS. Departmental NVRs remain the system of record for video. We consume RTSP and HLS only.

## 2. Overall architecture

Three lanes. (1) Official Sentinel/Corp8 grid: cameras, ingest hub, password portal, RTSP on 103.250.160.189:8554, HLS on cctv.corp8.cloud. (2) Sentinel Command: React UI + FastAPI + JSON store + JPEG evidence. (3) AI pipeline: FFmpeg/OpenCV → YOLOv8n → RapidOCR.

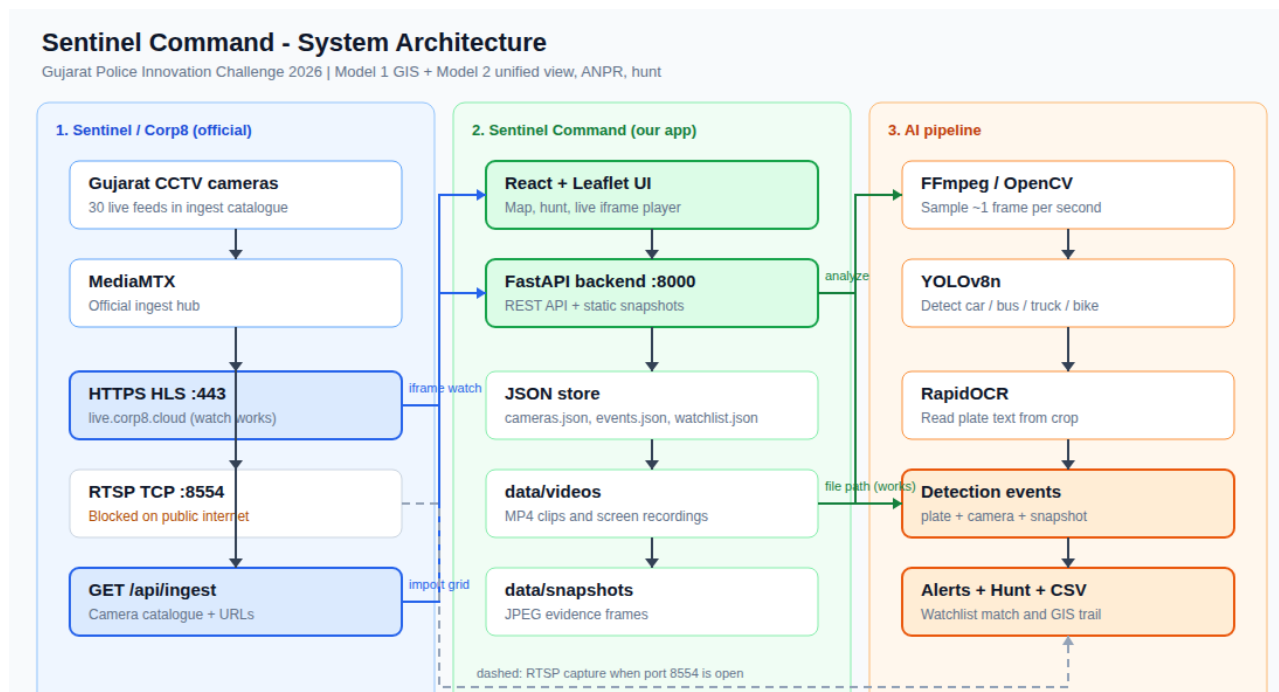


Figure 1. System architecture — official grid (left), our app (centre), AI (right).

## 3. Integrating heterogeneous cameras

We do not speak every vendor SDK in this prototype. The sandbox already normalises feeds to RTSP and HLS. Our integration contract is the integrator guide on [cctv.corp8.cloud/resource](https://cctv.corp8.cloud/resource):

| Step          | Mechanism                                                                                                                           |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Authenticate  | POST /auth/login; HttpOnly cookie held in a server CookieJar. Password in .env, never git.                                          |
| Catalogue     | GET https://cctv.corp8.cloud/cameras.json — array of {id, name} for cam01...cam30.                                                  |
| Watch         | HLS https://cctv.corp8.cloud/camXX/index.m3u8 proxied as /live/{id}/index.m3u8 with key + segments.                                 |
| AI ingest     | RTSP TCP rtsp://103.250.160.189:8554/stream/camXX.<br>OPENCV_FFMPEG_CAPTURE_OPTIONS=rtsp_transport;tcp. PTS timing. Backoff 2s–30s. |
| Second system | Local MP4s registered as cameras (Gate, Paldi, Visat recording) — Model 2 “two systems” proof.                                      |
| Onboarding    | Bulk CSV import, POST /cameras, and POST /cameras/sentinel/import.                                                                  |

Existing departmental VMS/storage are unaffected: we never record 24-hour video, never publish into the gateway, and never require replacing NVRs.

### 4. Live stream ingest and processing

Watch path and AI path are deliberately different. HLS on 443 is for humans and restricted networks. RTSP on 8554 is for OpenCV. Playlists on the sandbox are AES-128 VOD loops presented as live; the UI seeks currentTime = now % duration, matching the official portal.

Capture samples ~1 JPEG/s for a short window (default ~12s). Times come from stream PTS (or clip position for files), not CAP\_PROP\_FPS. Decoder join warnings (missing POC) are not fatal. We pace load: only the selected camera is captured.

### 5. Watchlist correlation and alerts

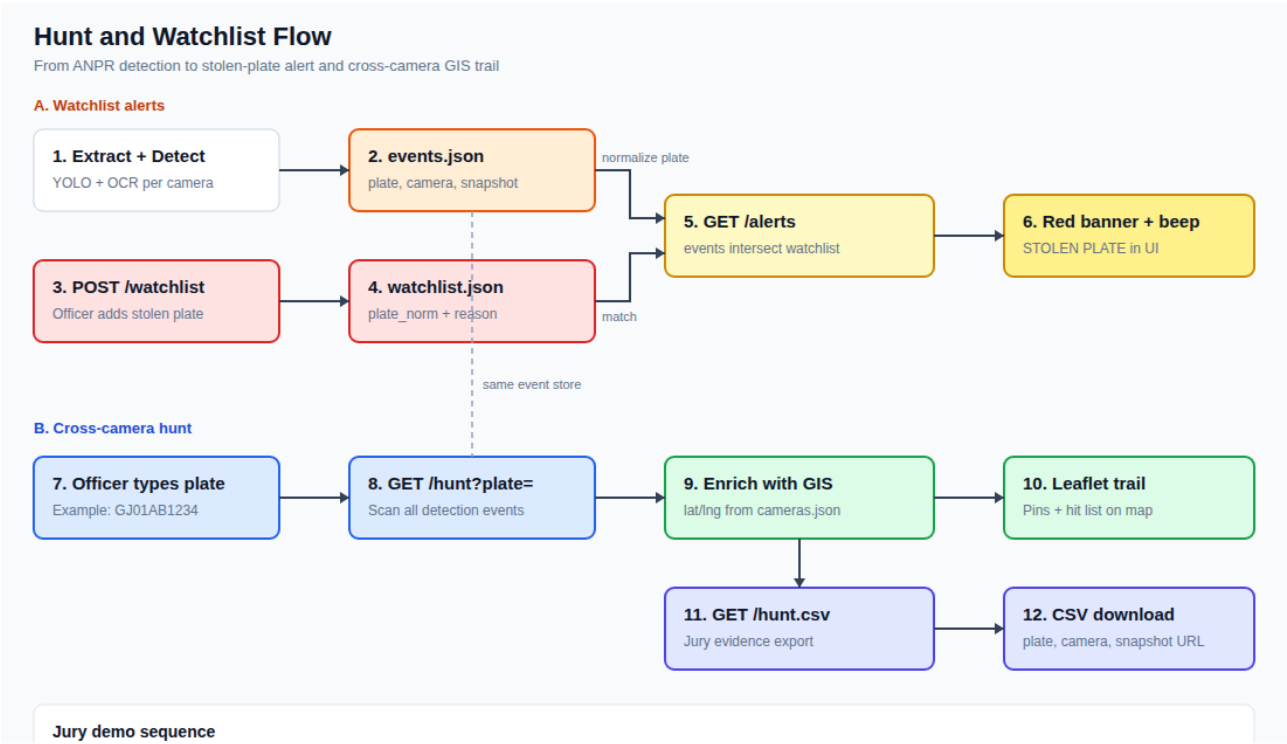


Figure 2. Watchlist intersection and cross-camera hunt.

The brief allows a representative watchlist. Items live in watchlist.json (plate\_norm, reason). Each ANPR event is normalised (non-alphanumerics stripped, uppercased). GET /alerts is the intersection of events  $\cap$  watchlist — not a separate store. UI: red STOLEN PLATE banner + beep. VAHAN / SARTHI / eGujCop are designed as later HTTP adapters behind the same normalize + match interface; they are not live in this prototype.

Hunt: GET /hunt?plate= scans all events, sorts by captured\_at then t\_sec, attaches lat/lng from the registry, draws a Leaflet polyline, and exports GET /hunt.csv. If the jury plate was not yet detected, the operator captures 4–6 road cameras, detects, and hunts again.

## 6. AI-powered video analytics

| Stage    | Technology                        | Output                                               |
|----------|-----------------------------------|------------------------------------------------------|
| Decode   | FFmpeg / OpenCV                   | JPEG stills under data/snapshots/{camera_id}/        |
| Detect   | Ultralytics YOLOv8n (nano, CPU)   | Boxes for car / motorcycle / bus / truck             |
| Read     | RapidOCR ONNX on lower 45% of box | Plate string + score                                 |
| Annotate | YOLO plot + plate overlay         | Evidence JPEG in annotated/                          |
| Index    | events.json                       | camera, time, label, plate, snapshot URL, confidence |

Face recognition, crowd counting, and intrusion zones are out of scope for this solo build. Measured: Paldi official cam04 RTSP sample produced 61 vehicle events and 0 readable plates (junction too distant). Own-feed Gate/Paldi clips produced 13 hits on GJ01AB1234. Both results are shown to the jury — we do not claim OCR on every statewide PTZ.

## 7. Alert workflow

Prioritisation in this prototype is binary: watchlist match vs not. Visualisation is the banner, hot event rows, and hunt trail. Interaction: add/remove plates, hunt, export CSV, open snapshot. Production would add severity, acknowledgement, and audit — listed as Phase 2.

## 8. Scalability, interoperability, security, performance

### 8.1 Scale path (~80,000 cameras)

| Phase         | Change                                                                  |
|---------------|-------------------------------------------------------------------------|
| Hackathon now | JSON store, one node, 30–40 cameras, CPU YOLO, short samples.           |
| Phase 2       | PostgreSQL + PostGIS, Redis watchlist, object storage, simple RBAC.     |
| Phase 3       | Zone RTSP workers, queue, horizontal API, GPU batching, HLS edge cache. |
| Phase 4       | VAHAN/eGujCop adapters, optional FRS, HA/DR, regional PoPs.             |

Principle: keep raw video near source; centralise metadata and alerts. Do not transcode 80k streams in one hall. Bandwidth: ANPR workers pull RTSP only for cameras under active analytics; officers use HLS. Storage: hot (days of snapshots), warm (object store), cold (department NVR retention 7–15 days as today).

### 8.2 Interoperability

Open protocols only (RTSP, HLS, REST JSON). Catalogue is the contract. New cameras appear by re-fetching cameras.json or CSV. Vendor lock-in is avoided by not embedding a proprietary VMS.

### 8.3 Security

Consume-only. Grid password not in source control. Session cookie server-side. No public write APIs. CORS limited to local UI origins. Production needs SSO/RBAC, TLS everywhere, audit logs, network segmentation — documented, not

implemented in the sandbox app.

## 9. Component map and APIs

| Module                                           | Responsibility                                          |
|--------------------------------------------------|---------------------------------------------------------|
| main.py                                          | REST, CORS, /media mounts, /live HLS proxy route        |
| sentinel.py                                      | Login, cameras.json, URL rewrite, TCP probe, cookie jar |
| store.py                                         | Registry CRUD, GIS enrich, CSV, gap report              |
| live_capture.py                                  | RTSP TCP sampling with PTS; HLS fallback                |
| live_proxy.py                                    | Playlist/key/segment proxy for the browser player       |
| ingest.py                                        | FFmpeg frames from MP4                                  |
| detect.py                                        | YOLO + OCR + annotated JPEG + events                    |
| events.py / watchlist.py / alerts.py / plates.py | Index, stolen list, intersection, plate norm            |

Key endpoints: GET /health, /cameras, /cameras/gap, /cameras.csv, POST /cameras/sentinel/import, POST /sentinel/login, POST /cameras/{id}/extract, POST /cameras/{id}/analyze, GET /events, POST /watchlist, GET /alerts, GET /hunt, GET /hunt.csv, GET /live/{id}/index.m3u8.

UI: <http://localhost:5174> (Vite proxies to FastAPI :8000). OpenAPI: <http://127.0.0.1:8000/docs>.

## 10. Prerequisites from departments

To onboard a real department we need: RTSP or HLS URL (or ONVIF device URL for a future adapter), approximate lat/lng, owning department, camera type, and whether 8554/443 is reachable from the analytics zone. Official lat/lng in the catalogue would replace our name-based geocode. Watchlist APIs (VAHAN stolen vehicles) need a documented JSON contract and credentials.

## 11. Known limitations (for evaluators)

Sandbox catalogue is ~30 cameras, not 50. ANPR is on-demand short samples, not a continuous 30-camera worker. JSON persistence, not PostGIS. No RBAC. Live Paldi OCR often fails on distance. These are stated so the working hunt, GIS, official ingest, and own-feed plates can be trusted.

This HLD accompanies the working prototype, the solution presentation, the own-feed and government-feed recordings, and the analytics output report in docs/jury-pack/.